

CSCE 671 HUMAN-COMPUTER INTERACTION · MODULE 04 · LECTURE NOTES

Mental Models

What the user believes the system is.

MODULE OUTCOMES

Explain the three models and the gaps between them.

Explain how mental models are formed.

Elicit a user's mental model.

Design a conceptual model and convey it.

Diagnose mistakes as model mismatches.

1 Three models

Model	Whose it is	Where it lives
The design model	The designer's.	In their head, and in the code — neither of which the user can inspect.
The system image	Nobody's — it is the artefact itself.	The interface, the documentation, the vocabulary, and the observable behaviour.
The user's model	The user's.	Inferred entirely from the system image, plus whatever analogies and conventions they bring (§2).

The designer communicates with the user only through the system image — there is no other channel — so a perfectly coherent design model conveyed by a poor system image produces a wrong user model, and the design was still wrong. The only-through-the-system-image point is this module's core, and it is why 'but the model is coherent if you understand it' is not a defence.

THE USER WILL BUILD A MODEL WHETHER OR NOT YOU GAVE THEM ONE

People do not operate a system without some theory of it — they construct one from the interface, the vocabulary, the observed behaviour, and whatever analogy comes most readily to hand.

So the question is never whether they have a model but whether it is the one you intended — and **if you did not design one, they will build something out of the accidents of your implementation**, which is reliably worse than anything you would have chosen.

And a wrong model produces mistakes rather than slips (Module 03 §4) — **confident, correctly-executed, wrong actions** — **which are harder to catch than slips and considerably more damaging**, because the user has no reason to check.

Which means designing the conceptual model is a design activity in its own right — **and it comes before the layout**, because the layout's job is to convey it (§3). A project that starts with screens has skipped this step and will convey whatever the screens happen to imply.

2 How models form

- **Analogy with something already known** — a folder, a desk, a conversation, a document, a queue. **Powerful, and it imports the analogy's properties whether you wanted them or not** (see the callout).
- **The vocabulary. Calling something 'archive' implies it remains retrievable; 'remove' implies it is gone** — and **the word does more work than any amount of help text**, because it is present at the moment of decision and the help text is not (Module 01 §2).
- **The observed behaviour**, including the timing, the orderings, and the error cases — **people theorise from what the system actually does**, and they are quite good at it given enough exposure.
- **And other systems' conventions, which is why consistency with the platform is worth more than internal elegance**: the user arrives with a model already, and matching it is free.
- **So the vocabulary is the highest-leverage design decision in this module, and it is usually made in a hurry** by whoever wrote the first version of the code — which is worth correcting deliberately later, even though renaming is disruptive.

AN ANALOGY IMPORTS PROPERTIES YOU DID NOT CHOOSE

A 'folder' implies that a file is in exactly one place — that is what folders are — so a tagging system presented as folders produces confusion at exactly the point where the analogy breaks: the moment something appears in two.

And the breakage is worse than having used no analogy at all, because the user was confident right up until the moment it failed, and now distrusts the parts of the analogy that were working — which costs them everything they had learned.

So choose the analogy for where it breaks, not only for where it fits — and signal the break explicitly at the point where it occurs, which is a design requirement rather than a documentation one.

Or choose a weaker analogy deliberately: a term with no strong prior implication forces the user to learn the actual model, which is slower and more accurate — and is the right trade for a system whose behaviour genuinely does not match anything familiar.

3 Conveying it

- **Make the state visible.** A system whose state is hidden cannot convey a model of its state — the user has nothing to infer from — **which describes most configuration systems** (Module 10 §4) and most background processes.
- **Make the structure visible.** If the data has a hierarchy, show the hierarchy; and if it does not, do not **imply one** with indentation or nesting that the model does not support (§2's folder problem).
- **Make the consequences visible before the action rather than after it** — a preview is a **model-communication device at least as much as a convenience**, because it shows what the system believes the action means.
- **And keep the vocabulary consistent and accurate** — **one name per concept, and the name should be true**: a 'save' that uploads, or a 'delete' that archives, teaches a wrong model very efficiently.
- **Which together make the model inferable** from the interface alone — and **documentation is the fallback for when the system image failed. That is not an argument against documentation and is an argument against relying on it**, since Module 01 §2's non-reading finding applies.

4 Eliciting it

ASK THEM TO EXPLAIN THE SYSTEM TO YOU, AND THE MISMATCH IS THE FINDING

Ask a user to describe what happens when they do something, before they do it — and the difference between their account and the truth is a design defect with a location, which is far more useful than a general impression of confusion.

And ask them to draw it, which surfaces structural beliefs that verbal description omits — whether they think things are nested, shared, copied, or linked, which is exactly the class of belief that produces Module 03 §4's mistakes.

Or ask them to predict: 'what will happen if you click that' is the sharpest single question in this course, because a wrong prediction is an unambiguous finding requiring no interpretation and admitting no argument.

Which is Module 07's technique applied to beliefs rather than to actions — and it finds the mistakes, where observing actions finds the slips. A full study wants both, and the prediction question costs nothing to add.

- **Change the vocabulary first, because it is the cheapest change with the largest effect (§2)** — and because a wrong name actively teaches the wrong model.
- **Then make the relevant state or structure visible (§3), so that the model is inferable rather than taught** — which is more robust, since it works for users who arrive later and read nothing.
- **Then consider changing the system itself** — **sometimes the user's model is better than yours, and that is a legitimate outcome of the investigation** rather than a failure of it.

- **And only then write documentation, which is the weakest of the four options and the one usually reached for first** — because it is the only one that does not require changing the product.
- **Which is the ordering that makes this module actionable. 'Sometimes the user's model is better than yours' is worth taking seriously: a consistently wrong expectation across five users is evidence about the domain rather than about them**, and it has changed real product designs for the better.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Norman — The Design of Everyday Things, chapter 3	The whole module — the three models, the system image, and the knowledge-in-the-world argument.
Johnson-Laird — Mental Models	§1 and §2's cognitive basis — where the term comes from and what it is claimed to be. Library copy.
Kuang & Fabricant — User Friendly	§2's analogies, historically — how particular metaphors were chosen and what they cost later. Library copy.
Nielsen Norman Group on mental models (free)	§4's elicitation, practically — with the interview protocols.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- 1 **Write out the design model** for something you built, in a paragraph.
- 2 **Ask somebody to explain the same system** and compare.
- 3 **List everything your system image conveys** that you did not intend.
- 4 **Find an analogy in software you use** and the exact point where it breaks.
- 5 **Find a name in your own software** that implies something false.
- 6 **Rename it** and note what else has to change.
- 7 **Ask three people to draw how a system works**, and compare the drawings.
- 8 **Ask five people to predict** what a button will do, before clicking.
- 9 **Find a case where the users' shared wrong expectation** is better than your design.
- 10 **Apply Part 4's four fixes in order** to one mismatch.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Name the three models and where each lives.
- 2 Why is the system image the only channel?
- 3 Why will the user have a model regardless?
- 4 What kind of error does a wrong model produce, and why is it worse?
- 5 Name four sources of a user's model, and the highest-leverage one.
- 6 Why is a broken analogy worse than none?
- 7 Give four ways a system image conveys a model.
- 8 What is documentation's role, honestly?
- 9 Give three elicitation techniques and the sharpest question.
- 10 Give the four fixes in order of preference.